

SENAI – Hermenegildo Campos De Almeida

ETAPA 1

CONFIGURAÇÃO, INVESTIGAÇÃO DE PORTAS E MODELAGEM DO SISTEMA

ERICK PINHEIRO DE MELO

SAMUEL SILVA PAULINO

IGOR IAGO DE JESUS GONÇALVES

PROFESSOR: WILLIAM

Guarulhos
2025

SUMÁRIO

1. Configuração do ambiente.....	3
2. Investigação das portas funcionais.....	4
3. Modelagem do sistema.....	6

1. CONFIGURAÇÃO DO AMBIENTE (ESP8266)

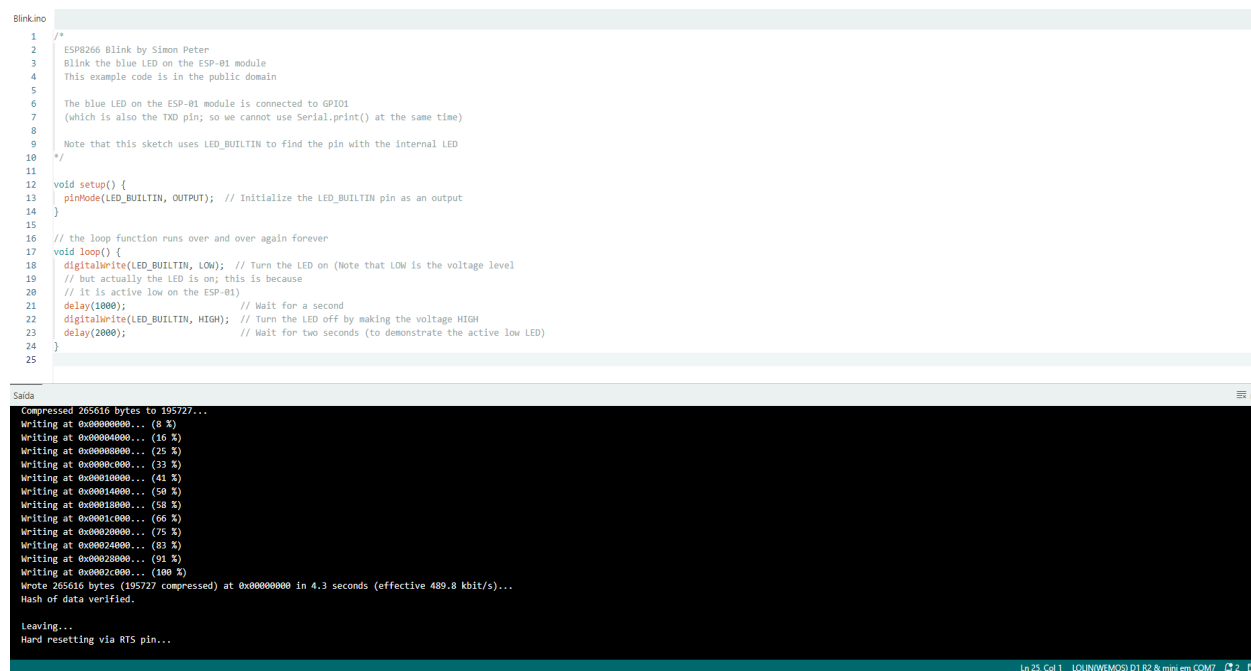
No primeiro passo, realizamos a configuração da IDE para programar o ESP8266 sem problemas. Pra isso, começamos acessando a aba “Arquivo” e depois “Preferências”. Nessa parte, adicionamos a URL

http://arduino.esp8266.com/stable/package_esp8266com_index.json, que é necessário pra que a IDE reconheça a placa ESP8266.

Depois disso, vamos até a aba “Ferramentas”, clicamos em “Placa” e em seguida em “Gerenciador de Placas”. Então buscamos por “ESP8266” e fazemos a instalação. Esse processo é importante porque adiciona suporte para programar esse tipo de placa dentro da IDE.

Depois da instalação, selecionamos a placa que estamos utilizando, que no nosso caso foi a **LOLIN (WEMOS) D1 R2 & Mini**. Também configuramos a porta de comunicação, que apareceu como COM7, mas esse número pode variar dependendo da porta USB que você conectar, então não precisa ser exatamente igual.

Com tudo configurado, fizemos um teste básico utilizando o código “Blink”, que é para verificar se a placa está funcionando. Esse código faz o LED da placa piscar, indicando que a comunicação entre o computador e o ESP8266 tá funcionando. Por último, realizamos o upload do código e deu tudo certo, como pode ver no print a seguir:



```
Blink.ino
1  /*
2   * ESP8266 Blink by Simon Peter
3   * Blink the blue LED on the ESP-01 module
4   * This example code is in the public domain
5   *
6   * The blue LED on the ESP-01 module is connected to GPIO1
7   * (which is also the TXD pin; so we cannot use Serial.print() at the same time)
8   *
9   * Note that this sketch uses LED_BUILTIN to find the pin with the internal LED
10  */
11
12  void setup() {
13    pinMode(LED_BUILTIN, OUTPUT); // Initialize the LED_BUILTIN pin as an output
14  }
15
16  // the loop function runs over and over again forever
17  void loop() {
18    digitalWrite(LED_BUILTIN, LOW); // Turn the LED on (Note that LOW is the voltage level
19    // but actually the LED is on; this is because
20    // it is active low on the ESP-01)
21    delay(1000); // Wait for a second
22    digitalWrite(LED_BUILTIN, HIGH); // Turn the LED off by making the voltage HIGH
23    delay(2000); // Wait for two seconds (to demonstrate the active low LED)
24  }
25
```

```
Compressed 265616 bytes to 195727...
Writing at 0x00000000... (8 %)
Writing at 0x00004000... (16 %)
Writing at 0x00008000... (25 %)
Writing at 0x0000c000... (33 %)
Writing at 0x00010000... (41 %)
Writing at 0x00014000... (50 %)
Writing at 0x00018000... (58 %)
Writing at 0x0001c000... (66 %)
Writing at 0x00020000... (75 %)
Writing at 0x00024000... (83 %)
Writing at 0x00028000... (91 %)
Writing at 0x0002c000... (100 %)
Wrote 265616 bytes (195727 compressed) at 0x00000000 in 4.3 seconds (effective 489.8 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
Ln 25, Col 1 LOLIN(WEMOS) D1 R2 & mini em COM7
```

2. INVESTIGAÇÃO DAS PORTAS FUNCIONAIS

- Podem ser utilizados como entrada: D2, D3 e A0
- Podem ser utilizados como saída: D5 ao D13
- Possuem restrições: D3, D9, D10, D11 e A0
- Devem ser evitados: D9, D10 e D11

1-Fonte utilizada: a única fonte utilizada foi o ChatGPT

2-Processo de investigação:

3-

PINO	PODE USAR?	TIPO	RESTRIÇÃO	JUSTIFICATIVA
D2	SIM	ENTRADA	NÃO	BOTÃO FUNCIONOU
D3	CUIDADO	ENTRADA	INICIALIZAÇÃO	PODE INTERFERIR NO BOOT
D5	SIM	SAIDA	NÃO	BUZZER FUNCIONOU
D9	EVITAR	SAIDA	COMUNICAÇÃO	INSTABILIDADE
D10	EVITAR	SAIDA	COMUNICAÇÃO	INSTABILIDADE
D11	CUIDADO	SAIDA	POSSIVEL CONFLITO	FUNCIONOU, MAS CUIDADO
D12	SIM	SAIDA	NÃO	LED FUNCIONOU
D13	SIM	SAIDA	NÃO	LED FUNCIONOU
A0	SIM	ENTRADA	ANALÓGICO	LEITURA DE ROTAÇÃO

4- Evidências

Nem todos os pinos podem ser usados. Por que?

Alguns pinos possuem funções específicas, como boot e comunicação interna. Esses pinos precisam estar em estados lógicos corretos, caso contrário o sistema não funciona.

O que pode acontecer se um pino inadequado for utilizado?

A placa pode não iniciar, entrar em modo incorreto, reiniciar continuamente, travar ou apresentar falhas na comunicação e no funcionamento dos componentes.

Como isso impacta um sistema real de automação?

Leitura incorreta de sensores, falhas de comunicação e até a parada do sistema, comprometendo a confiabilidade e a segurança da automação.

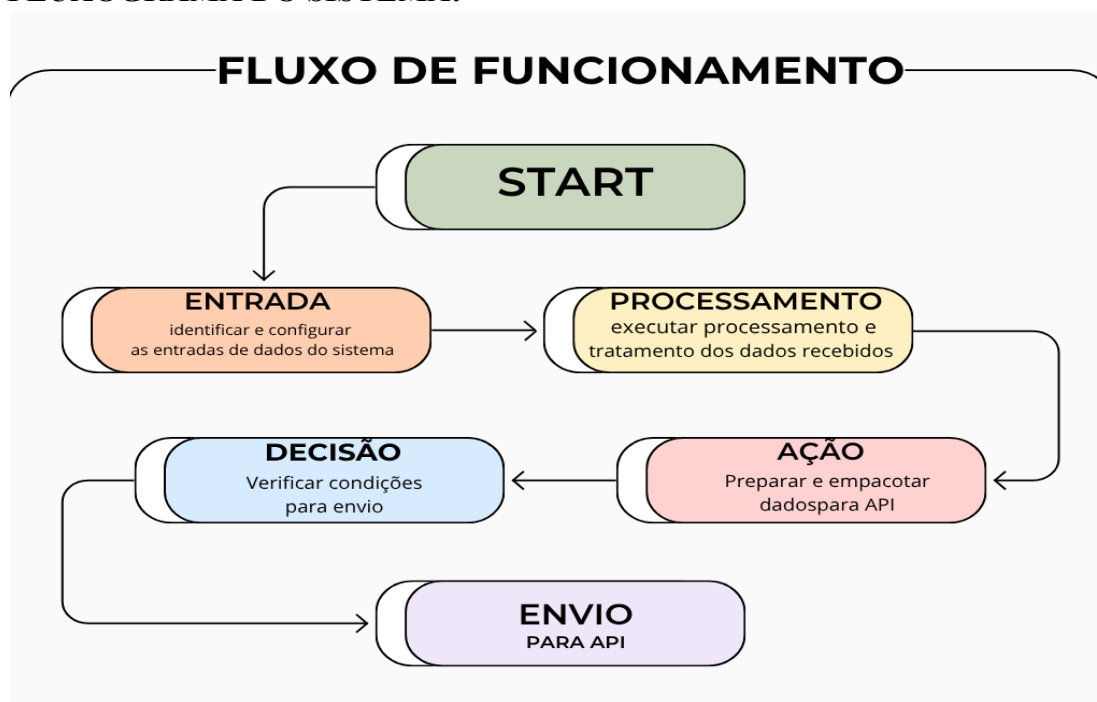
3. MODELAGEM DO SISTEMA

A modelagem define como o sistema deve reagir às entradas. Essa etapa é importante porque organiza a lógica antes da programação final. Variáveis principais: temperatura, umidade, rotação, estado do sistema, botões e comando IR.

REGRAS DE FUNCIONAMENTO:

CONDIÇÃO	ESTADO	AÇÃO
BOTÃO PRESSIONADO	COMANDO MANUAL	ALTERAR ESTADO DO SISTEMA
TEMPERATURA PERIGOSA	ALERTA TÉRMICO	ACIONAR BUZZER E LED VERMELHO
UMIDADE ABAIXO DO LIMITE	AMBIENTE SECO	SINALIZAR E REGISTRAR
ROTAÇÃO FORA DO PADRÃO	FALHA OPERACIONAL	AVISO VISUAL E SONORO
COMANDO IR RECEBIDO	CONTROLE REMOTO	EXECUTAR AÇÃO

FLUXOGRAMA DO SISTEMA:



CONCLUSÃO

A etapa inicial do projeto é fundamental para garantir que o sistema tenha base técnica sólida. A configuração correta do ambiente, a identificação dos pinos seguros e a definição das regras de funcionamento reduzem erros e aumentam a confiabilidade do desenvolvimento.

Com essa estrutura, o grupo passa a ter um modelo funcional e organizado

para avançar às próximas etapas do projeto, como integração com API, armazenamento em banco de dados e monitoramento remoto.